

FPGA-based Acceleration for High Frequency Trading

by

Yile ZHENG

A Thesis Submitted to
The Hong Kong University of Science and Technology
in Partial Fulfillment of the Requirements for
the Degree of Master of Philosophy
in the Department of Electronic and Computer Engineering

January 2023, Hong Kong

Authorization

I hereby declare that I am the sole author of the thesis.

I authorize the Hong Kong University of Science and Technology to lend this thesis to other institutions or individuals for the purpose of scholarly research.

I further authorize the Hong Kong University of Science and Technology to reproduce the thesis by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

Signature redacted

Yile ZHENG

January 2023

FPGA-based Acceleration for High Frequency Trading

by

Yile ZHENG

This is to certify that I have examined the above MPhil thesis
and have found that it is complete and satisfactory in all respects,
and that any and all revisions required by
the thesis examination committee have been made.

Signature redacted

Prof. Wei ZHANG, Thesis Supervisor

Signature redacted

Prof. Andrew Wing On POON, Head of Department

Thesis Examination Committee

- | | |
|---------------------------------|---|
| 1. Prof. Wei ZHANG (Supervisor) | Department of Electronic and Computer Engineering |
| 2. Prof. Xiaomeng LI | Department of Electronic and Computer Engineering |
| 3. Prof. Zhiyao XIE | Department of Electronic and Computer Engineering |

Department of Electronic and Computer Engineering
January 2023

Acknowledgments

Time flies. Along with the endless pandemic, I shall come to the end of my study. I am much grateful for being one of the members of the Reconfigurable Computing Systems Lab, where Prof. Zhang and my lovely lab mates accompany me all the way witnessing part of my life at HKUST. Many thanks to Prof. Zhang for your support of my research interest, and for making all this happen. Thank my friendly lab mates for all your help either in my life or my research. And most great thanks to my family for your unconditional care and love. Thank all of the people who come along with me, making this precious memory. Goodbye to HKUST.

TABLE OF CONTENTS

Title Page	i
Authorization Page	ii
Signature Page	iii
Acknowledgments	iv
Table of Contents	v
Abstract	vii
Chapter 1 Introduction	1
1.1 High-frequency Trading	1
1.2 FPGA Acceleration in HFT	3
1.3 My Works	4
Chapter 2 Local Order Book Reconstruction	6
2.1 Introduction	6
2.2 Background Knowledge	8
2.2.1 Order Book	8
2.2.2 Tick Data	8
2.2.3 Order Book Reconstruction	9
2.3 Implementation Details	10
2.3.1 Simplifying the Complexity of the Task	10
2.3.2 Order Book Data Structure	11
2.3.2.1 Differences from a Hash Table	13
2.3.3 Order Operations	13
2.3.4 Memory Interfaces	15
2.4 Experiments and Analysis	16
2.4.1 Experiment Setup	16
2.4.2 Resource Usage	17
2.4.3 Latency	19
2.5 Conclusion	19
Chapter 3 An Time-series Predictive Model and its Acceleration	21
3.1 Introduction	21
3.2 Related Works	21
3.3 Methods	23
3.3.1 Inspiration	23
3.3.2 Network Architecture	23
3.3.3 Training Steps	25
3.3.4 Hardware Acceleration	26

3.4 Results and Analysis	30
3.4.1 Analysis of the Predictive Model	30
3.4.2 Performance of the Acceleration	34
3.5 Conclusion	35
Chapter 4 Conclusion and Expectation	36
4.1 Conclusion	36
4.2 Future Work	37
References	38
Appendix A Order Book	41

FPGA-based Acceleration for High Frequency Trading

by Yile ZHENG

Department of Electronic and Computer Engineering

The Hong Kong University of Science and Technology

Abstract

High-frequency trading is a highly competitive area, where extremely low latency of every module in the trading system is pursued so that the traders can seize the market opportunities at the first moment. FPGA (Field Programmable Gate Array) becomes the best solution concerning low latency and reconfigurability. This work explores the FPGA implementations of two modules in the system: the local order book reconstructing system as part of the data handling module, and the hardware-accelerated predictive model for the trading strategy module. These two modules usually act as the key components of the system, and the improvement in their latency can bring great profit.

The local order book reconstructing system is proposed to obtain remarkable memory usage and operating latency balance, which dramatically reduces the memory usage by 1.98 to 93.21 times while maintaining the latency at a low level. Different PS-PL interfaces in the Xilinx UltraScale+ MPSoC are also explored in this application, which shows similar performance among them.

The predictive model, which shows lower prediction error and execution time, is also proposed to reveal the possibility of implementing a time-consuming neural network model in HFT. This model is inspired by the interpretation of the market microstructure. The input price signal is decomposed into the event signal and the system transfer function representing the market impact of the event signal, which enable the creation of this interpretable model. It is designed to learn the decomposing ability and the market impact patterns at the first training, and fine-tune the predicting branch next. The

model shows low prediction error and explainable behaviors, which make it more reliable to traders. Then the specific hardware acceleration method is applied to the well-trained model, which speeds up the execution time by up to 6.63 times and enables it for a higher-frequency scenario.

CHAPTER 1

INTRODUCTION

We are in a new era when the development of science and technology influences all walks of life. The automation of software programs is also well adopted in the financial industry, where traders use algorithms to complete the trading automatically without the invention of human beings. It is named algorithmic trading. And the underlying trading logic used for making trading decisions is called a trading strategy. The algorithm is less emotional so it can analyze the market more objectively compared to human beings.

1.1 High-frequency Trading

The types of algorithmic trading can be categorized by their executing frequency. While some trading strategies screen price movement within the day and focus on the daily prices of stocks, some strategies look into the real-time price and finish the trade in the day. Daily strategies are classified into long-, mid- and short-term strategies depending on the frequency their make transactions. And those that trade several to thousands of times within a day are classified as high-frequency trading (HFT). Beyond the HFT, there is also a further categorization according to the interval from an hour to a nanosecond. But the boundary is blurred. To simplify, in the thesis, we refer to strategies that retrieve market data in an interval lower than 1 minute when talking about HFT. And the system needs to have real-time updated market data, while the trading decision being generated in a minute-wise manner.

A typical trading system is usually constructed as shown in Fig. 1.1. It is a closed loop where data from the market are fed in and trading orders come out. Firstly, an HFT system connects directly or indirectly to the exchange, a broker, or a data provider, receiving data broadcast from the data source.

The network module receives network packets from the Ethernet and de-encapsulates the packets for downstream module use, while it also receives order generated from the

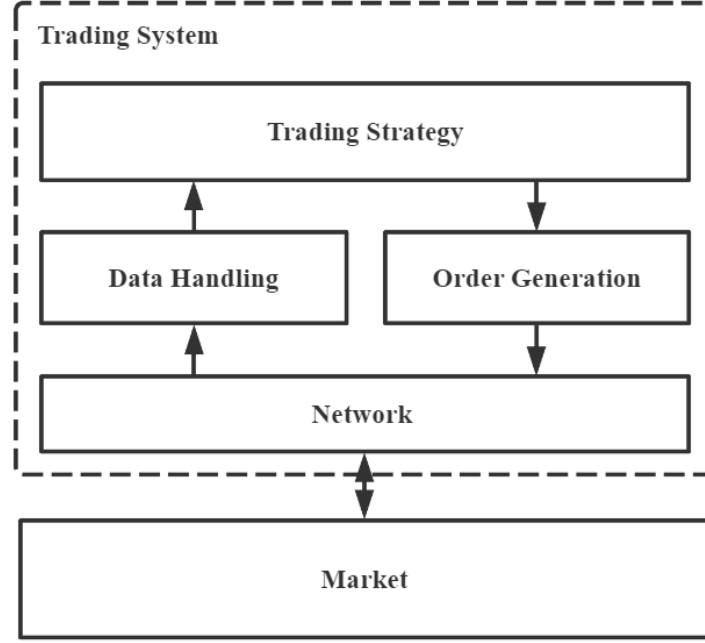


Figure 1.1: A general framework of an HFT system

upstream modules and encapsulates it to send to the exchange through the Ethernet. Following is the data handling module which takes the order and trade information decoded by the network module and applies pre-processing algorithms to generate more meaningful data for different kinds of trading strategies. For example, commonly, it aggregates the randomly arriving tick data into periodic data, such as microsecond-wise, second-wise, minute-wise candle stick data, etc., or reconstructs the order book of the exchange locally, or generates indicators like moving average line, trade volume, Bollinger bands, and other technical indicators according to the need of the downstream trading strategy.

Afterward, there is the decision-making trading strategy module. The profit-making trading strategy analyzes the real-time market data and generates orders at the time when it figures out it is a good timing through its trading logic. The trading strategy can be any kind of algorithm, such as a mathematical model, simple if-else logic, machine learning, neural network model, and so on, as long as it makes profits.

The last part, the order generation module, receives trading signals released by the trading strategy and generates and tradable order for sending to the exchange. This part usually also contains pre-trade risk management check and portfolio management to make sure that the fast-running strategy makes a correct decision and stops the potential loss in the case that the strategy generates abnormal orders.

1.2 FPGA Acceleration in HFT

In the real application of HFT, the institutions always want their system to be in a low latency mainly having to aims. The first one is to make sure the system is reacting to the most up-to-date market date. The market price always changes from time to time. When the price goes to the profitable level, the system wants to react and make a order right away. But if the latency of the system is too high, at the time when the order is placed, the opportunity has already gone and the order is executed at a worse price which brings loss. So a low-latency system is desired to guarantee that the system is reacting to the newest market quote. Another scenario is that, when a trading opportunity appears, other traders discover it and place the order as well. However, the orders of the counterparty are not enough to fill all these orders. So they are filled in a first-come-first-served manner. In this circumstance, traders want their system having lower latency than others' so that their orders can be filled prior to anyone else.

As the HFT market becomes more and more competitive, HFT institutions fully devote themselves to reducing the latency of their trading systems and networks, so that they can seize the market opportunities before others. To shorten the latency of the system, the field programmable gate array (FPGA) comes on the stage as the first option because of its intrinsic low latency and reconfigurability features, as well as its high throughput benefiting from the hardware parallelism. Institutions such as Citadel, Optiver, etc., have already developed their own low-latency FPGA HFT systems, supporting their large amount market-making or arbitrage strategies. And there are also FPGA service providers, like Maxeler Technology, Velvettech, Algo-logic, etc. Most commonly, ad-hoc FPGA acceleration is implemented for the network and data handling module in the system. Because these parts are usually fixed and generally used. For example, in a modern PC, a hardware NIC (network interface card) handles the first two layers (physical, data link layers) in the 5-layer TCP/IP network stack model, and software cores process the rest. This introduces unacceptable latency mainly from the PCIe data transfer from the NIC to the CPU and software network protocol processing. However, an FPGA network module in the HFT system can be designed to handle all the network encapsulating and de-encapsulating protocols on the same chip, cutting down the data transfer time and algorithm execution time. It dramatically improves the network handling latency making sure that the downstream modules receive the most up-to-date data as soon as

possible. After that, the FPGA data handling module pipelines the operations and starts processing the data once it arrives. It can parallelly generate different indicators or run different data handling algorithms at the same time, which is a remarkable feature that a CPU is not able to achieve, or can only achieve at a great cost of expensive running core resources.

Except for the above common implementations, the FPGA design can be applied to any part of the HFT system. Some institutions that pursue extremely low latency implement the entire closed loop on the FPGA without involving a CPU, or just use the CPU for parameter configuration, which greatly reduces the low latency of the trading system and enables them to catch a trading opportunity the first time it appears.

1.3 My Works

As one of the applications of the data handling module, the local order book reconstruction algorithm is an essential part of the trading system, which is commonly used by the industry. An efficient and low-latency local order book reconstruction implementation saves previous execution time for the trading loop, making sure that the trading system is reacting to the most up-to-date market quote, which is a key goal of HFT and leads to considerable profits. However, the order books usually take up a large memory size as there are thousands of stocks in the market. And each of them needs to have a mirror local book. Here we face the time and space balance problem. In the second chapter of this thesis, a low-latency order book reconstruction with balanced time and resource consumption is proposed. It achieves a low latency while at the same time greatly reduces the memory consumption giving credit to its flexible data structure and dynamic memory allocation.

The trading strategy in the system is usually considered as the heart of the trading, and even the heart of the institution. Because the algorithm makes trading decisions which are where the profits come from. In the HFT institutions, there are usually a specific research department working on developing competitive trading strategies. However, as quick reaction to the market is the primary goal in HFT, complicated algorithms that require more execution time such as neural network models are not preferred. In the third chapter, a FPGA-accelerated time-series predictive neural network model is proposed. The model is designed based on the high-frequency data, inspired by the theory of signals and systems

and neural networks. It predicts the movement of the price given the historical price time series. Then it is implemented on FPGA to accelerate the computing and adapt it to the HFT scenario. The model shows interpretable behaviors and low prediction error, while the FPGA implementation shorten its execution time and reveals the possiblily of the adoption of the neural network algorithm in HFT.

The organization of the thesis is as follow. The first chapter of the thesis illustrates the background of HFT and the 2 parts of my work, followed by a chapter that elaborates on the first part, order book reconstruction. And the details of another part of the work, a FPGA-accelerated predictive model, is in Chapter 3. Finally, Chapter 4 is organized as the conclusion of the thesis.

CHAPTER 2

LOCAL ORDER BOOK RECONSTRUCTION

2.1 Introduction

As the very first part where the HFT system receives first-hand market information, the ability to quickly handle and interpret from the most real-time tick data is essential for every HFT institution. An HFT institution that receives and interprets the market quotation before others, has a bigger probability to seize the opportunity of making profits. Order book reconstruction (OBR) is one of these applications. It syncs the order book in the exchange by making use of the order messages in the real-time tick data, which enables close access to the most up-to-date order book. In order to make sure the local order book is updated once receiving the order message, the order book operation latency should be as short as possible. But on another hand, there is a trade-off with memory resources as most of the problems have. There are thousands of stocks in the market, and each stock has its own order book. If the data structure of the order book occupies too large a size in the memory, it will end up with very considerable memory usage to implement a number of books for each stock.

While it is common to implement the order book reconstructing algorithm in the industry, there is a limited number of articles we can find about the topic. People in this industry choose to keep this business secret. In Huang's paper [1], he/she proposed a tick-level dataset LOBSTER and an OBR algorithm implemented with the object-oriented programming language, which is not targeting the online HFT system scenario, but for the research of the dataset. Conghui He, Wayne Luk, et al. [2] proposed several algorithms. The typical algorithm as said is based on the AVL tree, which maintains the depth of the tree to the same level so as to limit the maximum tree searching latency. However, the algorithm is not hardware friendly and is implemented on a CPU. The latency is much high than an FPGA implementation and longer than the interval of two consecutive order messages, which I measured from the US exchange data at the peak hours is around 1 microsecond. They also proposed another hardware-friendly data structure and algorithm,

named hardware-friendly order book update (HFOBU) algorithm, which dramatically reduce the latency to the order of hundred nanoseconds. But all the proposed designs are targeted at the China stock market, which has a fixed price range of $\pm 10\%$ open price. So they implement a fixed tick order book meaning that it reserves a slot for every price within the range, which is not robust to other markets and wastes large memory space because it is unlikely that the orders are distributed across the whole range at a specific moment. And the memory occupancy changes with the price. For instance, the close price of the Kweichow Moutai Co., Ltd. is 1771.00 on 23rd Dec. 2022. And the minimum price change is 0.01. Then the number of slots that should be reserved for one single Kweichow Moutai stock can be calculated as follow, which is a large number as there are thousands of stocks in the market.

$$Slot\# = \frac{1771.00 * 0.2}{0.01} = 35420$$

In order to reduce the reading time, they proposed a parallel reduction which parallelly compare prices step by step to get the top price. Then mask the result and perform another round of comparison to get the second top price. And repeat these operation for 5 times to get the top five levels. It is obvious that the reading latency will be long and it also increases along with the size of the preserved book. So they proposed another model with a cache to store the top five prices to reduce the read latency.

Another paper by Fu, et al. [3] proposed a CPU-FPGA hybrid table, which keeps a number of the most frequently used levels in the BRAM on the FPGA and the rest in the CPU AVL tree. The FPGA-side table is maintained by the bitonic sorting routine [4] which has exponential time complexity. So the FPGA cache size cannot be too large, while a small FPGA size also introduces extra data transferring latency between FPGA and CPU.

The OBR module proposed in the chapter obtains a better balance between resource occupancy and latency. It is implemented on the PL side of the Xilinx ZCU102 edge device designed by high-level synthesis (HLS). The book is stored in the PS side memory rather than the BRAM on the FPGA, because the BRAM resources are far from enough for multiple order books. The proposed algorithm has the following advantages:

- The memory usage is competitively low, which makes it scalable for a large order book system consisting of thousands of stocks.

- It obtains balanced and relatively low latency.
- It is robust to different markets.

This chapter is organized into 5 sections. This section is an introduction, followed by the background knowledge that is important for understanding the scenario of the work. After another section illustrates the implementation detail of the system, the experiment setup and result analysis section reveals the performance of the system. And at the end, there is a conclusion for this chapter.

2.2 Background Knowledge

In order to make trading decisions according to the market situation, traders first need to retrieve real-time market data from a data source provider to analyze based on the most up-to-date information.

2.2.1 Order Book

An order book is a place where the exchange stores the orders that are placed by the market participants but not yet traded. The full order book in the exchange is structured as one bid book (buy side), and one ask book (sell side). Each book contains a number of price levels. Each price level stores a number of orders of different order IDs at the same price. The bid book is sorted in decreasing order. The highest price on the top is the optimal price the buy side offers. And the ask book is sorted in increasing order. The optimal offer is the lowest price on the top.

2.2.2 Tick Data

The tick data, differing from the other data low-frequency traders use, refers to the messages issued from the exchange that contain the details of every trade and every order made to the exchange. Streaming tick data enable traders to follow up on the most recent change in the order book inside the stock exchange. It consists of order messages and trade messages.

Order messages are generated to indicate the change of the order book inside the exchange. When a new order from the market participant arrives at the exchange, it

will either match with other existing orders in the order book or be stored in the order book in case of no matching orders. If the incoming order matches one or several orders in the order book, the transaction happens, and it takes these orders out of the book, which means the order details of them are removed from the order book. At this moment, order messages are released to indicate these changes. In another case, if the incoming order cannot find any corresponding counterparty orders in the order book, it will be put into the book waiting for new incoming orders for a trade. Here another order message is broadcast to indicate this newly added order in the book. Except for the arrival of incoming orders, requests from the order owners to cancel the order in the book will also generate order messages.

Unlike order messages, trade messages are generated to indicate the occurrence of trades in the exchange. When the order matching takes place, it means that the deal between two participants is closed. However, it is not necessarily that orders from one side of the transaction of a trade should be from the order book. In-coming orders can match each other as well.

2.2.3 Order Book Reconstruction

While the full order book is maintained by the exchange, there is too much overhead regarding the duplicated information and the response delay if requesting the whole book each time during trading hours. So maintaining a local order book becomes essential for HFT institutions. They construct a local order book by interpreting the information of the order message in the tick data, which refers to the order book reconstruction (OBR). According to the order messages from the exchange, people can choose to maintain order books of different granularity locally in their system as needed. There are 3 types of local order books one can choose to maintain, which are shown in Fig.2.1.

- **Order Depth** The order details including order ID, price, timestamp, order size, etc. is maintained. Orders in each price level are organized in the correct order according to their timestamps.
- **Price Depth** The sizes of the orders are aggregated to their price levels, which builds the volume of each price level.
- **Top of book** Only the optimal prices of each side of the book are maintained.

bid volume	price	ask volume	...	bid II	bid I	price	ask I	ask II	...
	125	10				125	10		...
	118	10				118	10		...
	110	20				110	15	5	...
	102	20				102	10	10	...
	101	30				101	10	20	...
30	100		...	15	15	100			
10	97		...		10	97			
30	92		...	20	10	92			
30	87		...	15	15	87			
5	83		...		5	83			

bid price	Ask price
100	101

(a) Top of book

(b) Price depth

(c) Order depth

Figure 2.1: Three types of local order books to maintain

The order operations indicated by the order messages include several types, such as adding, canceling, modifying, trading, and so forth. Adding an order is indicating that a new order is not traded and added to the order book for later matching, while canceling an order indicates a removal of the order from the book. Modifying order refers to the change of the order size of an existing order initiated by the trader. And trading order also indicates the traded amount of the order, which can be partially or fully traded meaning a change or removal of the order respectively. Referring to the order book updates provided in the order messages, the OBR is feasible and even one of the essential applications of the HFT.

2.3 Implementation Details

In this section, the design details of the proposed order book is elaborated. We choose to implement a price depth order book, which in most of the cases is enough. The order sizes are aggregated to the price level. And the order operations are applied to the total price volume.

2.3.1 Simplifying the Complexity of the Task

There are several ticks I adopt in my design making the task simpler.

Order Book Operation Categorizing

While the operations indicated by the orders message from the exchange are quite various, I classified them into only 3 operations.

- **Add:** Insert a new order into the appropriate position in the order book.
- **Modify:** Change the order size of an existing order. This combines other operations, such as cancel order, trade order, which are substantially to modify the existing orders. For example, cancelling order is to change the order size to 0, while trading order is to deduct the traded amount from the order size.
- **Read:** To read a certain number of levels from the local order book.

So we only consider this three types of operations in the design.

Order Book Range

As some of the exchanges reject extremely high or extremely low orders. The prices in exchange's order book are almost placed within $\pm 5\%$ of the optimal price. This is a significant feature that decides the range of our order book. Let's call it the critical range hereafter.

In addition, in order to simplify the discussion, we only show the details of the ask side of the book in the following statement, because the the two sides are symmetric.

2.3.2 Order Book Data Structure

Firstly, I reshape the order book into a 2D array which has a fixed number of columns as shown in Fig.2.2. In order to make use of the fast indexing of the hash table and, at the same time, avoid the unnecessary memory occupancy, only the first block of each row is indexable, and the rest elements behind are linked. The shallow blocks in the figure lay out of the critical range. Row 65~69 will be including, when orders in Row 10~14 are all deleted and the rolling critical range slides down.

The above idea creates the data structure shown in Fig.2.3. Each row of the above 2D array converts to an ordered linked list (OLL), putting their first element in the indexable circular array (ICA) with a base index pointer and a optimal price maintained for index calculation. In the physical memory, ICA is implemented as a the fixed indexing space (FIS) which is accessed via an exact index calculated from the order price. And the rest price levels behind the first element in the OLL are all placed in the dynamic management space (DMS). To set up the book, there are three parameter we need to configure before the implementation: the maximum length f the OLL, the size FIS, and the DMS size. The

price	volume
10	50
12	100
13	75
15	55
16	15
17	30
18	10
19	30
21	30
24	5
27	15
...	...

(a) An example of order books

0~4					
5~9					
10~14	10		12	13	
15~19	15	16	17	18	19
20~24		21			24
25~29			27		
30~34				33	
.....					
	40	41			44
50~54		51			
55~59		56		58	
60~64					
65~69					

(b) The reshaped 2D arrangement of the order book.

Figure 2.2: Order book reshaping

maximum length of the OLL and the size of the FIS are customizable, while the size of the DMS are quite fixed according to the maximum number of price levels. The maximum length of OLL limits the maximum searching latency of the book. A larger maximum OLL length is suitable for the high price stock having sparser price distribution, while smaller maximum OLL length adapts to lower price stocks. And the size of the FIS is decided by the critical range together with the determined maximum OLL length. And since we can adjust the value of the maximum OLL length to adapt to the critical range, the size of FIS can also be fixed. Therefore, the memory usage of the system consisting of the FIS and the DMS can be limited in a low level regarding stock of different price, by selecting a appropriate maximum OLL length.

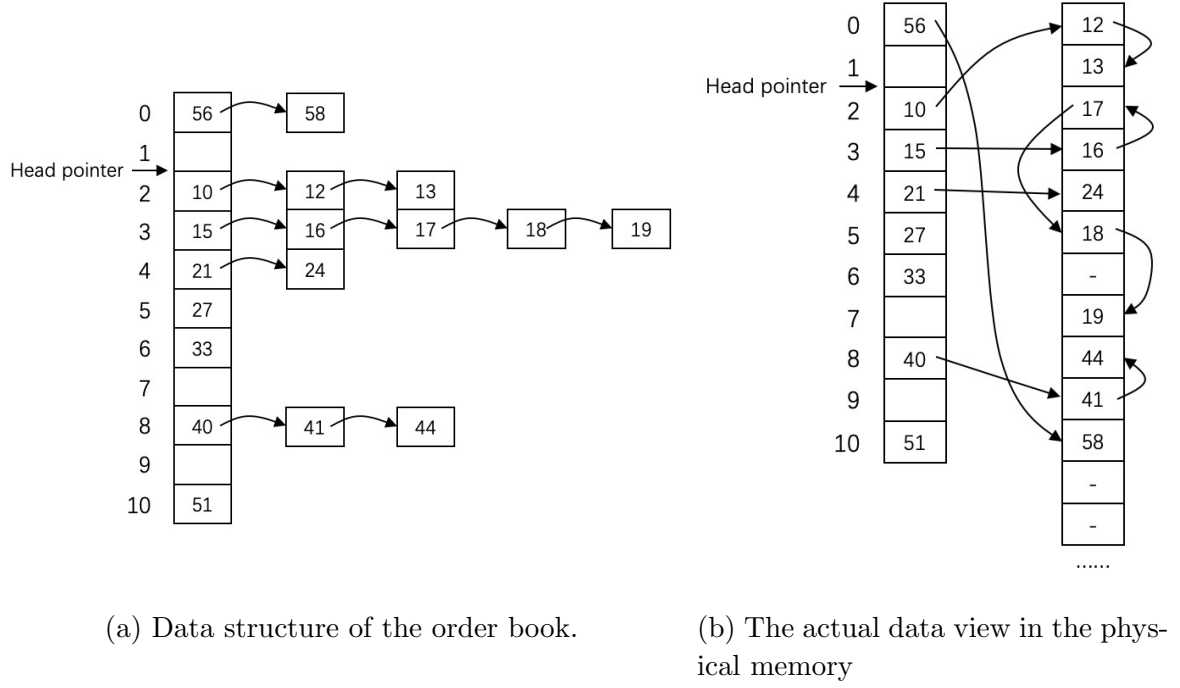


Figure 2.3: Three types of local order books to maintain

2.3.2.1 Differences from a Hash Table

Hash table directly calculates the index of the corresponding bucket according to the key and insert to the location. But in this proposed book, a head pointer needs to be maintained. And we need to calculate the offset first according to the price, then plus the base index to obtain the index value. And as hash table also introduces linked list to solve the collision, for this purpose, it usually does not manage the order of the values in the link. But the proposed design sorts the links as well as the buckets to make it easy to read the book. Besides, while the chain of collision can be very long in the hash table, our algorithm limits the maximum length of the chain to control the maximum latency.

2.3.3 Order Operations

The detail of operations to the order book are listed below.

Search

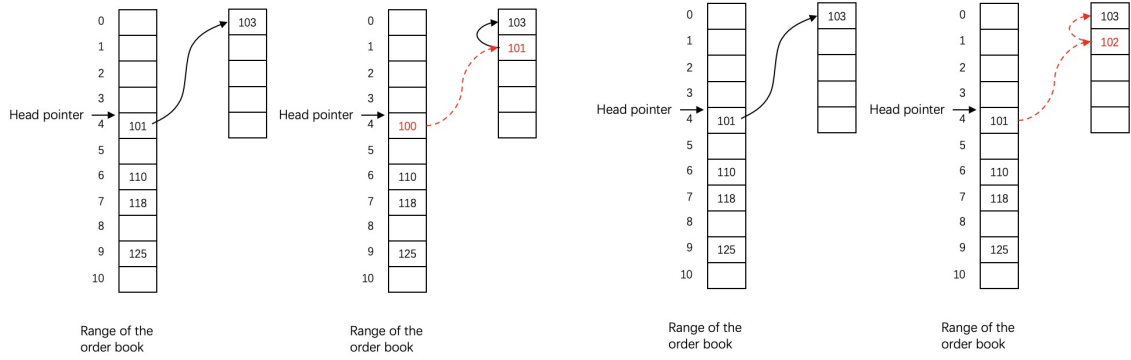
The search for the price consists of 2 steps. The first step is to calculate the index of where the OLL head locates on the ICA. Then search along the OLL until the target price is found. The expression of the index calculation is shown below.

$$offset = floor(\frac{price - optimal_price}{minimum_price_change * maximum_OLL_length})$$

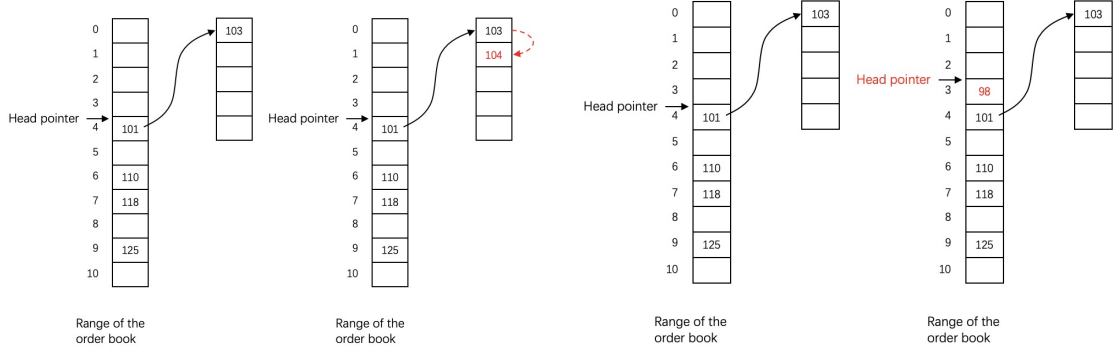
$$index = base_index + offset$$

Add

To add a new order to the book, the book first needs to search for the correct position on the OLL until the price of the read block is equal to or larger than that of the order, or otherwise reaches the end of the OLL. When there is no matching price, a new block needs to be created, as shown in Fig.2.4.



(a) Add a new block to the head of the OLL. (b) Add a new block in the middle of the OLL



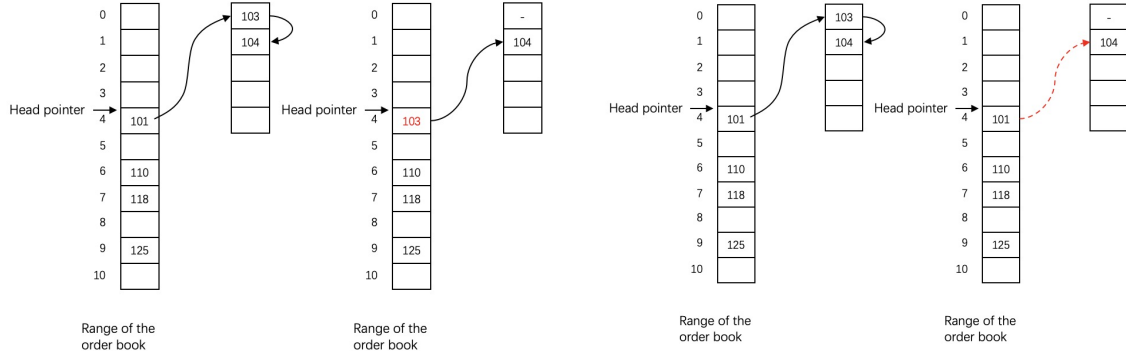
(c) Add a new block to the end of the OLL. (d) Add a new block of the more optimal price.

Figure 2.4: Three types of local order books to maintain

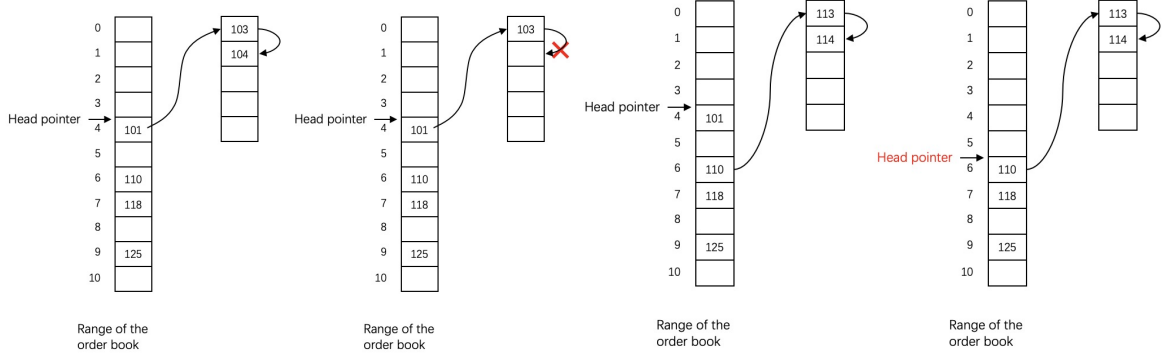
Modify

Modifying one of the blocks follows the same path as searching. The difference is that, when the volume of the price block is deducted to 0, it needs to remove the block from the OLL or sometimes from the ICA, as shown in Fig.2.5.

Read



(a) Add a new block to the head of the OLL. (b) Add a new block in the middle of the OLL



(c) Add a new block to the end of the OLL. (d) Add a new block of the more optimal price.

Figure 2.5: Three types of local order books to maintain

To read the content of the order book in an appropriate order, the read operation starts with the OLL at the base index of the ICA, then read along the OLL one by one till the end. Afterward, another OLL behind the last one is retrieved, and repeat the above steps until enough price levels are read.

2.3.4 Memory Interfaces

The design is implemented on the Xilinx Zynq UltraScale+ MPSoC. Because the book is located in the memory space of the PS, the memory access latency influences the latency of the system significantly. So multiple PS-PL data sharing interfaces are explored to see the performance. The connection of interfaces we are focused on is shown in Fig.2.6. There are 4 types of interfaces.

HPC

This is called a high-performance coherency interface. It connects to the system memory management unit (SMMU) before reaching the cache coherent interconnect (CCI). The SMMU executes the task of memory access permission check and translation from the

virtual memory address to the physical address. The HPC interface performs a one-way cache coherency to the L1 and L2 cache in the application processing unit (APU) via the CCI. But it has longer latency to access the DDR memory because it routes through the extra CCI before reaching to DDR interface.

HP

This type of interface, high-performance interface, access directly to the DDR memory without cache coherency, which facilitates high throughput block memory access and is the default connection model of PL acceleration kernels if developed in Vitis Kernel Flow.

ACP

The accelerator coherency port enables the PL to access the memory following the same path as the CPU does. The read and write access is under the management of the snoop control unit (SCU) within the CPU, which enforces the coherency of L1 and L2 caches in the CPU core.

ACE

Through this port, users can implement a one-way cache coherency without a cache in the PL side, or a two-way cache coherency with the hardware cache. In this thesis, the cached architecture is adopted to explore the feature of this interface. And the cache size is set to 32KB the same as the CPU L1 cache.

The detailed block design diagrams of the connection of the above interfaces are shown in Appendix A.

2.4 Experiments and Analysis

After determining the design, experiments and comparisons are set up to measure the performance. The details are elaborated on in the section

2.4.1 Experiment Setup

The implementations are all tested on the Xilinx ZCU102 edge device powered by a Zynq UltraScale+ MPSoC chip. And the PL kernel is driven by a clock of frequency of 200 MHz. A program running on the application processing unit (APU) powered by the Quad Cortex-A53 core is designed to feed the order messages to the order book kernel

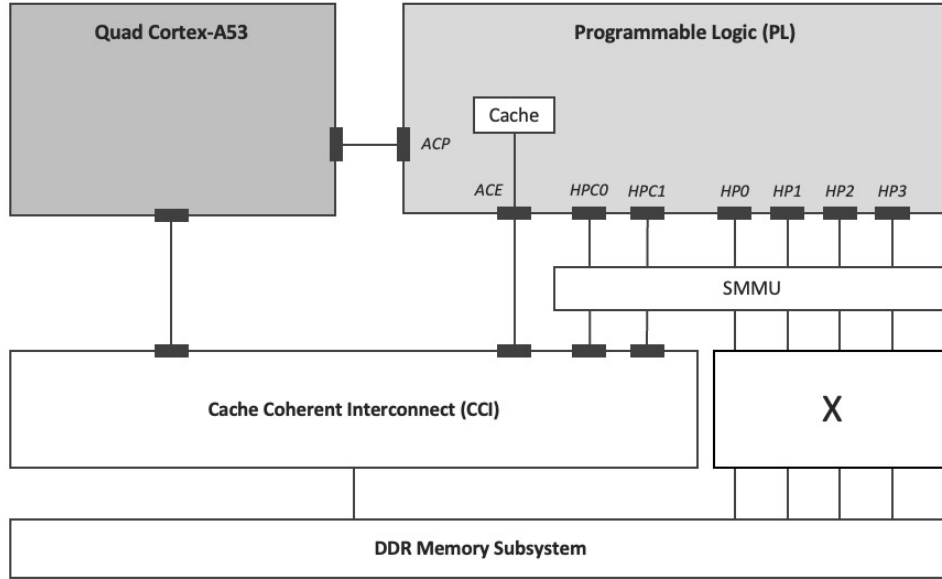


Figure 2.6: Architecture of some interfaces between PS and PL on the Xilinx Zynq UltraScale+ MPSoC

and measure the execution latency. The testing data comes from the tick data of the MSFT stock of date June 21st, 2012. And the tick by tick true order book contents is used for verification. The program records the start time and the end time respectively before and after the execution of the kernel for running time measurement.

As the designs, experiment environments and the metrics of the existing works vary from each other, we found the work of He et al. [2] comparable to our design. So comparison in terms of resources usage and latency is mainly made with their proposed hardware-friendly order book update (HFOBU) algorithm and HFOBU with a customized cache (HFOBU-C) which maintains an extra top-five prices cache in the BRAM to accelerate the read operation. Their design is running on a Maxeler MAX4 MAIA acceleration card with a Stratix-V 5SGSD8 FPGA at the clock frequency of 200 MHz as well and a 48GB DDR3. Please refer to the original paper for more details.

2.4.2 Resource Usage

In this section, a comparison of theoretical memory usage is conducted. However, because the HFOBU and HFOBU-C target the China stock market which has a fixed $\pm 10\%$ price range for each stock. To make the comparison reasonable, we put our design under the scenario of China market as well, although it has the additional strength of adapting to markets without the price range limit. Three stocks of price from low to high

are used to see the memory usage of the models along with the price level.

In addition, because the element in their data structure is similar to ours. Information such as price, volume, and link to the next element is included in both of our designs. We can safely assume that we are using basic elements of the same size. So the value of memory usage is calculated in the unit of blocks, which represent a certain size of memory needed for one element.

We selected three stocks of different prices from the China A-share market for this comparison. The detail can be found in the Table. 2.1

Stock	Price	Model(blocks)		Configuration(blocks)		
		HFOBU	Proposed	FIS	DMS	OLL max length
Bank of Ningbo	30.77	516	260	60	200	5
BYD Company Ltd	252.18	5044	450	250	200	10
Kweichow Moutai	1771.00	35420	380	180	200	100

Table 2.1: Comparison of theoretical memory usages.

From the table, we can see that by adjusting the value of FIS, DMS size, and the max length of OLL, we can adapt the book well in our design. As the result, the memory usage of HFOBU is proportional to the stock price, while we can keep that of our design under a relatively low level by adjusting the configurations. And as the price gets higher, this advantage becomes more significant. This is undoubtedly essential for the order book reconstruction system where order books of thousands of stocks are maintained at the same time.

Another highlight for the table is that, as the price of the stock goes up, the memory needed first increases and then decreases. It is because the minimum price changes of these stocks are all equal to 0.01, which is relatively large for low-price stocks than the high-prices. When the price is low, there are a limited number of prices within the critical range. As the price becomes higher, the critical range becomes wider, and the number of tradable price increases. We need to extend the FIS size as well to fit into the critical range, while the maximum OLL length does not increase too much because the distribution of prices is still similar to the lower-price stocks. As we move to higher price stocks, the existing prices become sparse within the critical range. We can set the maximum OLL length to a higher value while most of the time there will be only several prices in the OLL. Then the FIS size decreases leading to a decrease in memory usage of

the proposed model.

2.4.3 Latency

In this section, the comparison over latency design is conducted. In He’s paper [2], they only discussed reading the top five price levels. So we also test our model under the condition of reading only the five top price levels, while reading any number of price levels is feasible in our design. Tests over the aforementioned different interfaces are also conducted for the proposed design. Table. 2.2 shows us the results.

Latency (ns)	He’s		Proposed			
	HFOBU	HFOBU-C	HP	HPC	ACE	ACP
Add	96	288	199	207	210	221
Modify	96	276	192	188	189	190
Read	1186	132	569	566	572	574

Table 2.2: Comparison of order book operation latency

From the table, we can also say that the interfaces have equal performances for this application. And compared with the HFOBU model, our design has more balanced latency regarding different operations. The add and modify latency of HFOBU is shorter than us since it is achieved by directly indexing which is the fastest case. But compared with HFOBU-C which needs to maintain the data structure and links, our design outperforms theirs. In the cost of higher latency of add and modify operation, the HFOBU-C model maintaining an extra top-five prices cache in the off-chip BRAM accelerated the read operation dramatically. And because it is implemented on hardware, it only accelerates reading a fixed number of price levels. But offering a limited and fixed number of price levels is far from enough for real application scenarios. and a larger cache should introduce more add and modify latency in their design. But our design having a unified order book does not have this problem. In addition, the latencies of our design is acceptable and low enough for peak order message interval of the scale of 1 microsecond.

2.5 Conclusion

In this chapter, a less memory-consuming and low-latency OBR system is proposed. The comparison of performance with He, et al.’s design reveals that our design outperforms theirs with low memory usage and has competitive latency. Our memory

usage and operation latency balanced system suits the real application scenario well which streams order messages of thousands of stocks with a wide range of prices. And the low memory usage attribute of the proposed design make it scalable for a large order book system when thousands of books are stored. Additionally, different PS-PL data sharing interfaces are tested for this module, which as a result are having equal performance regarding this algorithm.

CHAPTER 3

AN TIME-SERIES PREDICTIVE MODEL AND ITS ACCELERATION

3.1 Introduction

As the heart of trading, an appropriate trading strategy can make great profits for financial institutions and hedge funds, which also makes it a top business secret that no one likes to share. A trading strategy is a well-designed program that makes use of wide range kinds of methods, like mathematical statistic models, natural language processing, machine learning, or neural networks, to automatically analyze the market and generate trading signals. However, in HFT, neural network models are less taken into account because of the latency introduced by its relatively heavy computing workloads. In this chapter, we designed an interpretable time series predicting neural network model based on minute-wise high-frequency data, and accelerate it with FPGA to shorten the latency enabling it for higher-frequency application scenarios. The proposed design has the following contributions:

- The prediction precision is high.
- The model is interpretable and, therefore, more reliable for traders.
- The computing latency is shorter which enables it for high-frequency scenarios.

3.2 Related Works

Financial time series prediction models are researched by many individuals and institutions. Some tasks predict the direction of the price change, which can be categorized into the classification problem, while some focus on the forecast of the value of the price or its return, the regression problem. For classification scenarios, in Huang's paper [5], the SVM is adopted, while the decision tree algorithm is tested feasible in Wu's paper [6]. Other machine learning methods such as KNN[7], ensemble learning[8] and so forth also

have been researched. For price value forecasting, more complex sequence-to-sequence (seq2seq) models are proposed. The most typical sequence-to-sequence (seq2seq) models include RNN[9], GRU[10], LSTM[11], transformer[12], which now usually act as part of more complex models in the encoder-decoder structure. Chung et al.,[13] and Qian et al.[14] proposed their predictive models based on the LSTM, while [15] and [16] researched on DNN models together with some data pre-processing methods, such as principal component analysis (PCA), autoencoder (AE) and Restricted Boltzmann Model (RBM). Other methods such as reinforcement learning [17] also has been adopted in solving this problem.

Chong’s paper [16] explored the application of several machine learning and neural network models together with different data pre-processing methods in predicting the price return of stocks in Korean stock market.

Models:

1. AR(10): An auto-regressive model with the input of 10 lagged historical data;
2. DNN: A multi-layer perceptron (MLP) with 2 hidden layers and activation function of ReLU;
3. ANN: replace the activation function of DNN with sigmoid function;
4. DNN-AR: the DNN model together with a AR trained on the residual of DNN output;
5. AR-DNN: the AR model together with a CNN trained on the residual of AR;

Data pre-processing methods:

1. PCA: principal component analysis;
2. AE: autoencoder;
3. RBM: restricted Boltzmann model;

The performance of each combination of the above models and data pre-processing methods is measurement with the root mean squared error (RMSE) metric. And the best one belongs to the combination of method AE and model AR-DNN, which is 0.9621.

3.3 Methods

3.3.1 Inspiration

The design is inspired by the market microstructure theory, which finds that the market price moving is driven by a series of events that impact the market in various time scales. These events can be news, block trades, market sentiments, or others that cannot be easily observed. Referring to the theory of signals and systems [18], we can regard the events as an input signal and event impact as the transfer function of the system, which represents the collective interpretation and reaction of the market participants to the event. When an input signal passes through the system, it is convolved by the transfer function and generates the output signal. So we use the convolutional layer to recover the price time series from its event signal. And the extraction of the event signal is implemented by the transformer decoder layer.

The training of the model acts as an automatic pattern recognition, which figures out the way of decomposing the market price time series to events signals and modeling the impact of the events in later time steps as transfer functions. The extracted transfer function is expected to be like a damping oscillation curve [19]. The impact of the event decays as with the time step, while it oscillates because of the resistance of the market.

3.3.2 Network Architecture

The proposed network consists of two parts as most of the time series model does - an encoder followed by a decoder. Additionally, it has a prediction branch which will be fine-tuned after other parts of the model are well trained and frozen. The training steps will be illustrated in the next subsection.

As shown in Fig. 3.1, the input time series first goes through the sinusoidal position encoding layer, and then is passed to the transformer encoder layers (TEL). Here multiple TELs are adopted parallelly processing one position-encoded feature map so as to extract different types of event signals. As of here, we obtained the event signals, which are passed to the decoder consisting of 3 convolutional layers (CL) and a prediction branch. The first CL is a group convolution, which convolves over each channel separately without merging them together to one single channel. This CL merges the feature of the extracted time series along the feature dimension. After that, its output is concatenated with the result

of the prediction branch when the model runs in predicting mode, or directly padded with zeros when the prediction branch is disabled in the first training step. Following is another CL acting as a system that reflects market reactions. The convolution is also grouped to make sure that each type of event signal has its specific type of impact to the market. In the end, the last CL merges signals along the channel side and generates the final output series.

The prediction branch enforces the prediction of event signals. Together with the impact of previous events, it makes more reasonable predictions. It consists of 3 fully connected layers, analyzing the feature of event signals and generating the predicted event signal of the following time steps. The hyperparameters of the model are determined by hyperparameter sweeps. The detail is shown in Table. 3.1.

Attention needs to be paid to the Concat/Pad layer here that the padding or concate-

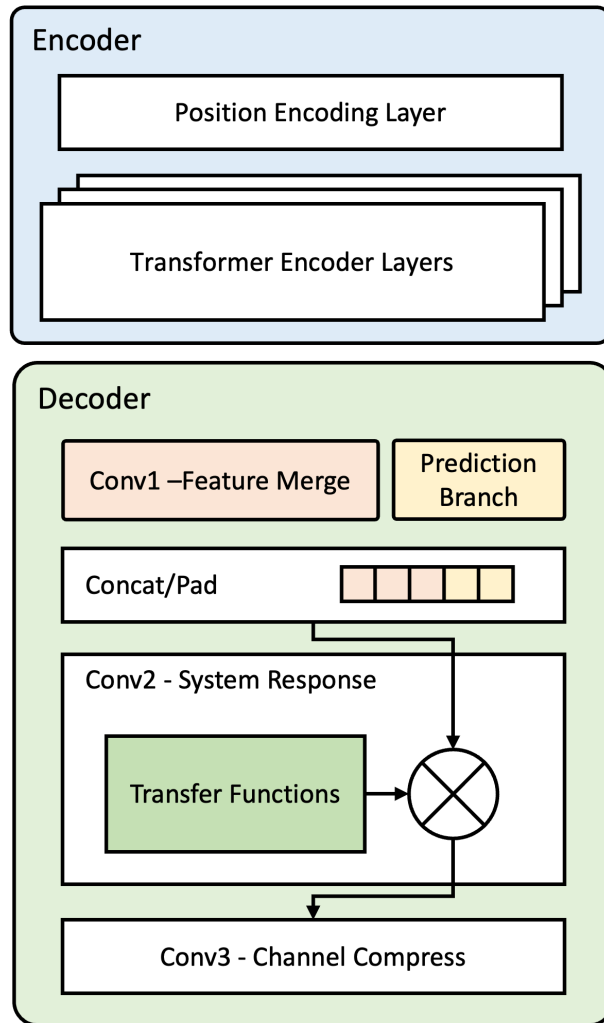


Figure 3.1: Structure of the proposed model

Layer	Detail	Shape
Input		(24, 4)
Position Encoding	Sinusoidal	(24, 4)
Transformer Encoder Layer X 4	head=4, Feedforward_size=4	(4, 24, 4)
Linear1	in=384, out=512	(512,)
Linear2	in=512, out=128	(128,)
Linear3	in=128, out=24	(24,)
Conv1	2D conv, kernel_size=(1, 4), channel_in=4, channel_out=4, group=4, padding=0	(4, 24, 1)
Concat/Pad	pad or concatenate the reshaped Linear3 output to the right of the Conv1 output, length=6	(4, 30)
Conv2	1D conv, kernel_size=7, channel_in=4, channel_out=4, group=4, padding=0	(4, 24)
Conv3	1D conv, kernel_size=1, channel_in =4, channel_out=1, group=1, padding=0	(24,)

Table 3.1: Detail of the proposed model

nation is only applied to the right of the series to complement the length of the future part of the output series as Fig. 3.2.

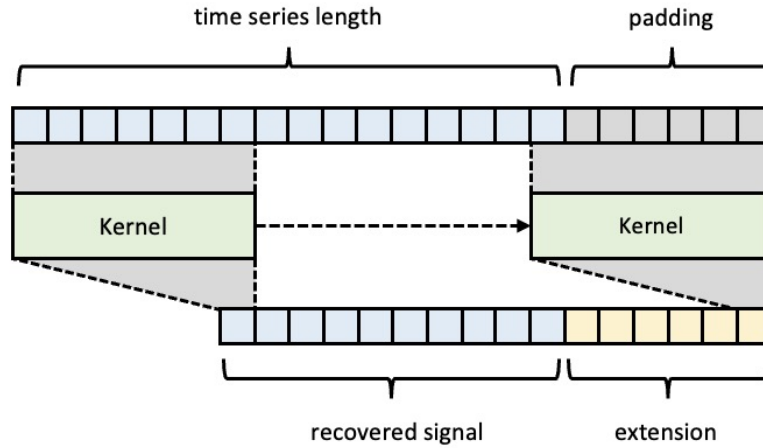


Figure 3.2: Details of managing the length of the series during the convolution.

3.3.3 Training Steps

The training procedure of this model is divided into two steps. While the first step enables the model to discover event signal and transfer functions of the market by itself, the second step training enables the prediction branch for fine-tuning and freezes the parameters of other layers.

First Step: Signal Recovering

In the first step of training, the task of the model is to recover the input signal. The

main goal of this step is to train the ability of the model to decompose the market time series event signals and reform the signal by the transfer functions. Fig. 3.2 shows the convolution operation of the Conv2 layer where the input and output series do not have equal lengths. The input and output series are right aligned in the design. The blue boxes of the input series indicate the output of Conv1, while the greys are padded with 0 to complement the length of the output so that we can have the extension of the impact of previous event signals as yellow boxes stand for. And the blue boxes of the output series represent the recovered signal which will participate the calculation of the loss function. The reason that the two series are aligned to the right is that the market is a causal system and it never sees future information. So only the segment of event signals that are prior to the discussed time step is included for the convolution. This detail makes the model reasonable.

What needs a mention is that although the extension segment is not included in the training, it reflects the extended impact of the previous market environment, which to some extent is also part of the prediction.

Second Step: Prediction Branch Fine-tune

For the second step of the training, the prediction branch is included, whose output will replace the padding mentioned above. While all the parameters trained in the first step are frozen, only the parameters of the prediction branch initialized by a Xavier uniform initializer [20] participate in the fine-tuning.

In this step, the loss is not from the model output, but the interim feature map, the event signal. The spirit is that the prediction branch is predicting the event signal, the most straightforward way is to directly compare the forecast event signal with what it should be when it is generated by the well-trained recovering model. Fig. 3.3 shows the detail. The time series on it are all signed by the time step. The recovering model on the right acts as the base model that generates truth event signals using future time series. Then the corresponding segment of them is taken by the loss function together with the forecasting segment from the under-training model for calculating the loss value.

3.3.4 Hardware Acceleration

After the model is fully trained and ready for deployment, we implement it on an FPGA to speed up the execution. While the structure of the model is elaborate in

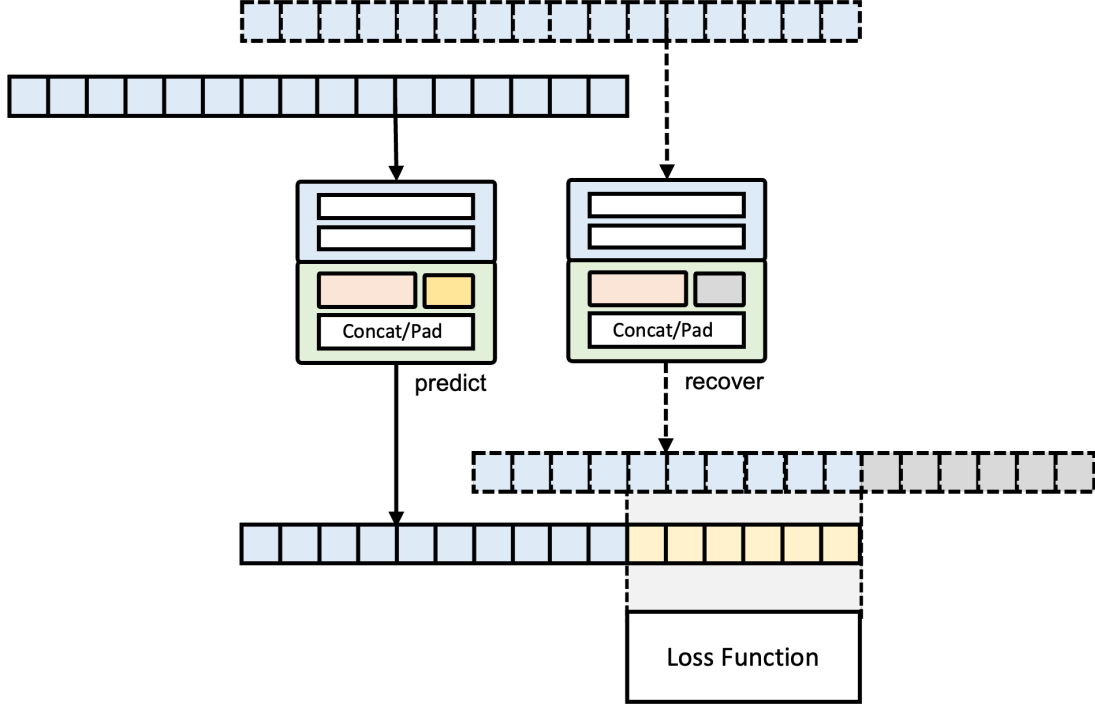


Figure 3.3: Compare the predicted segment of the time series with the truth generated by the base recovering model to calculate the loss for training.

Section 3.3.2 and the detail of the transformer encoder layer can be found in the original paper [12], We only discuss hardware optimization here. The major kinds of computation in the model are matrix multiplication and convolution, we applied some optimization methods to accelerate these computations. And except for the single operation, we also tried considering a series of operations as a whole and applying the optimized dataflow method. The details are illustrated below.

Dataflow-friendly Multi-head Attention

The multi-head attention layer consists of several consecutive operations, which are formulated as below:

$$Output = \text{concat}(head_1, head_h) * W^O$$

$$head_i = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{d_k}}) V_i,$$

where

$$Q_i = M^I * W_i^Q$$

$$K_i = M^I * W_i^K$$

$$V_i = M^I * W_i^V$$

$$d_k = d_{model}/h$$

While i indicates the attention head, W_i^Q , W_i^K , W_i^V , W^O are parameters of the model. M^I is the input matrix. And d_{model} is the dimension of the model input features. h is the number of heads.

So $\sqrt{d_k}$ is fixed when the architecture is determined and it is a constant scalar for the deployed model. Then we can reduce the scaling operation by multiplying it with one of the parameter arrays as shown below:

$$\begin{aligned} head_i &= \text{softmax}(\frac{M^I W_i^Q M^I W_i^K}{\sqrt{d_k}}) V_i \\ &= \text{softmax}(M^I W_i^{Qa} M^I W_i^K) V_i \end{aligned}$$

where

$$W_i^{Qa} = \frac{W_i^Q}{\sqrt{d_k}}$$

Besides, the dataflow-friendly design is also applied to this module. As explained in Fig. 3.4, the key of this method is to take into consideration the order of computing, so that the module takes the shortest time waiting for the first data needed from all the inputs. Multiple instances of the flow are created to parallelly process these heads.

Matrix Multiplication Optimization

The optimization of the matrix multiplication of our design refers to Dou, et al.'s work [21] and adapts to our design. The mechanism is shown in Fig. 3.5. We unroll

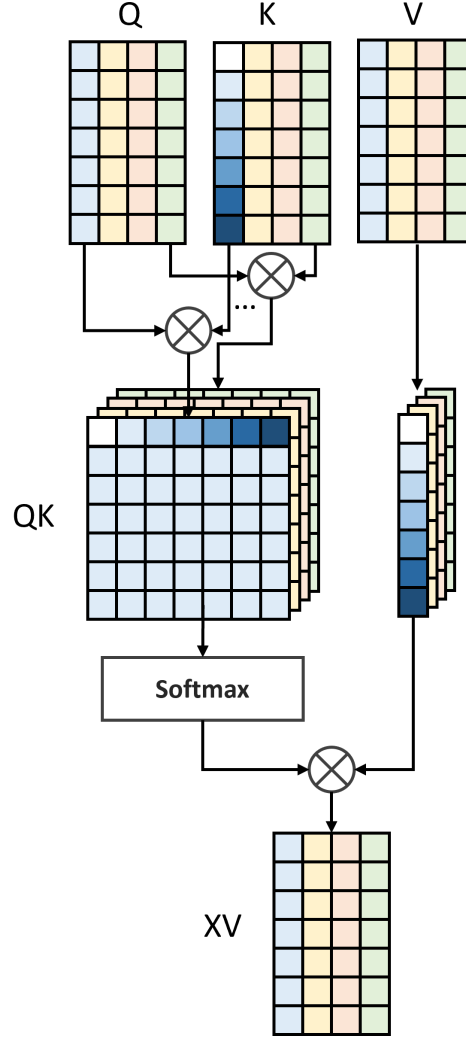


Figure 3.4: A dataflow-friendly design of the scaled dot-product multi-head attention

the multiplication and form a 4×4 computing matrix consisting of 16 multiplication-accumulation core. And the computing matrix iterates along the common dimension of two operands (horizontal dimension for the first operand, vertical for the second one). At the end of each iteration, a part of the output matrix is generated.

Convolution Optimization

Fig. 3.6 is the pseudo-code of the convolution operation, which has 6 nested loops. The variable b and k refer to the bias and weight parameter of the layer respectively. tin and $tout$ indicate the input and the output. And the number of input channels is represented by CI .

While unrolling the 3 inner loops shortens the iteration of calculating a single element of the output, unrolling the 3 outer loops extends the output size of a single iteration. To balance the fan-in and fan-out size and the memory access pattern of the input and

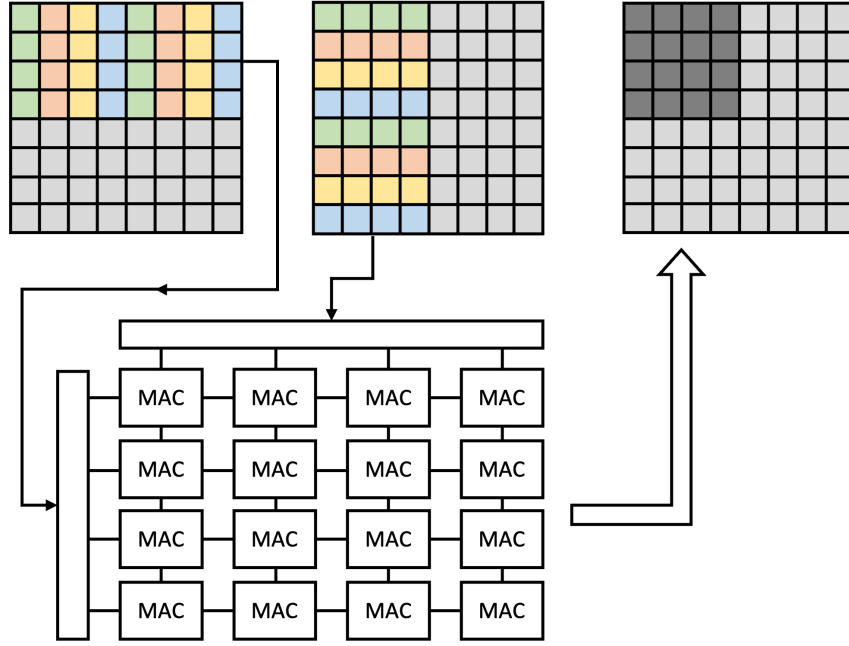


Figure 3.5: Matrix Multiplication implementation

output memory, we choose to unroll the 3 outer loops. We refer to the design of Ma, et al.'s [22] except that we take into consideration the situation of group convolution.

```

for co in CO:           // output channel
  for xo in XO:         // output X
    for yo in YO:       // output Y

      pricebase_t tmp = b[co];
      for kci in KCI:    // kernel channel in
        for kx in KX:    // kernel X
          for ky in KY:  // kernel Y
            tmp += tin[(KCI*co)%CI+kci][yo+ky][xo+kx] * k[co][kci][ky][kx];
      tout[co][yo][xo] = tmp;

```

Figure 3.6: Pseudo code of the convolution operation

3.4 Results and Analysis

3.4.1 Analysis of the Predictive Model

Experiment Setup

The proposed network model is trained and tested using the return rate of minute-wise OHLC (open, high, low, close) prices of 10 cryptocurrency tokens from the cryptocurrency exchange. We applied some pre-processing methods to the price return data to obtain

stationary signals. For example, the normalized log price return is generated for further use. The expression is shown below.

$$\log_return = \log(\text{price}_t / \text{price}_{t-1})$$

$$\text{norm_log_return} = \frac{\log_return - \text{mean}(\log_return)}{\text{std}(\log_return)}$$

Interpretability: First Step Training

During the process of training, the interim feature layers and the parameters of the Conv2 layer (transfer functions) are extracted for observation. Fig. 3.7 shows the input and the output feature maps of the Conv2 layer and its trainable parameters, as well as the comparison between the target and the model output.

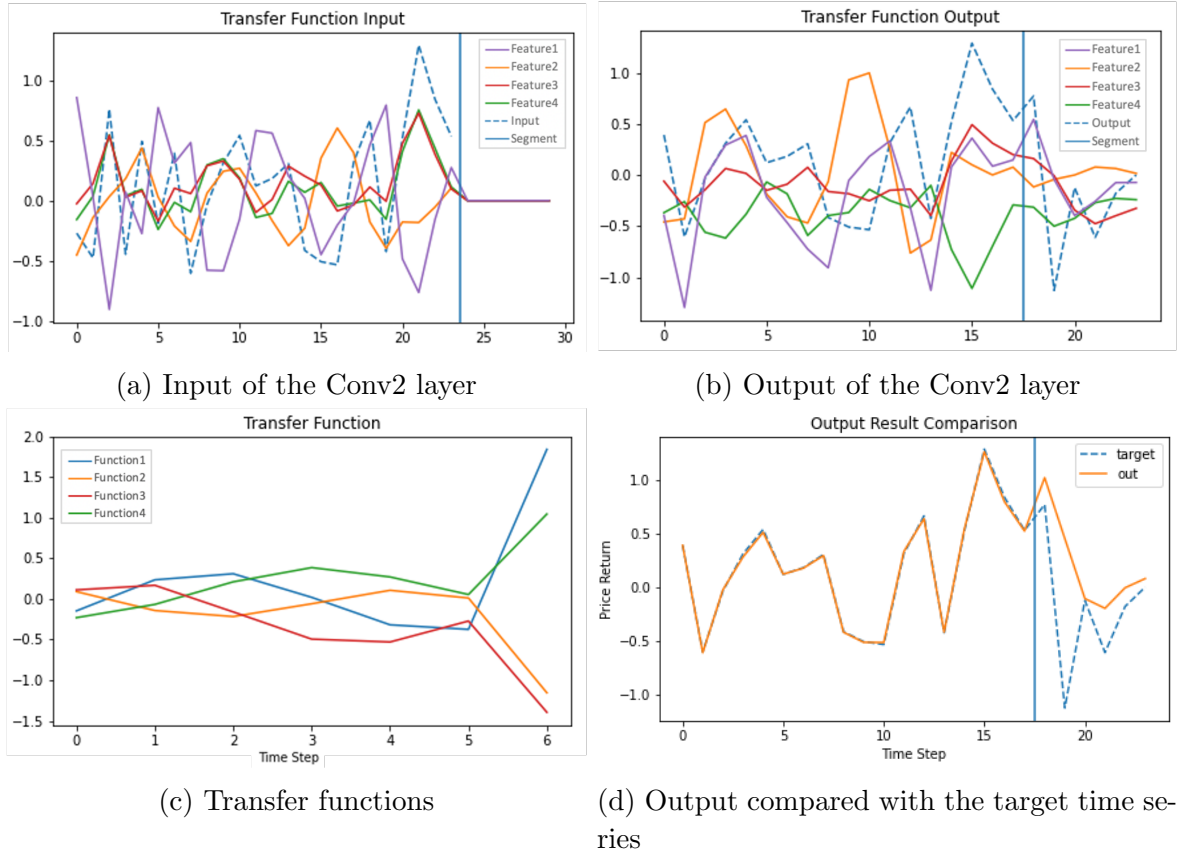


Figure 3.7: The behaviors of the model training in the first step

In the figures, the blue vertical lines separate the time series into the recovered signal segment and the extension segment as elaborated in Fig. 3.2. The blue dash lines represent the input time series for the sub-figure (a) and the target output series for (b) and (d). Signals in (a) reveal the event signals extracted by the decoder, while the extension segment to the right is padded with 0. After applying the transfer functions, the impact

of previous event signals is extended to the right side. (c) shows the transfer functions obtained by the training. Because the convolutional layer doesn't do a horizontal flipping of the convolutional kernel before the calculation as it should be for signal processing, the time order of the transfer functions should be from right to left here. The impact of the event signal starts with significant magnitude at the first time point to the right, and converges to a small value while oscillating, which matches exactly our assumption and expectation of a damping oscillation curve mentioned in Section 3.3.1. The model figured out a reasonable pattern to interpret the market. In the end, the output is compared with the target series in sub-figure (d). The objective of the first step of training is to recover the signal and enable the model to interpret the market by itself. The output recovered signal on the left side of the vertical line matches the target time series well, while on the right side the extended signal obtains the trend of the time series knowing the impact of previous event signals.

Interpretability: Second Step Training

In the second step of the training, we fine-tune the prediction branch to enhance the predictive ability of the model. The same features are retrieved from the model and put in Fig. 3.8. At this time, the extension segment is not padded with 0. Instead, the predicted event signal replaces it as shown in sub-figure (a). The predicted segment follows well a similar pattern as previous signals. The training at the step only aims at the prediction branch. So the obtained transfer functions are aligned with the first step. When we look at the prediction extension segment on the right side of the vertical line in sub-figure (d), the prediction matches the target well at the first few time steps while the accuracy decays for the later time steps.

Results and Comparison

Our model predicts the next 6 time steps. The performance of each time step is shown in Table. 3.2 for the model completing the first step training and Table. 3.3 for the final model.

Time Step	1	2	3	4	5	6
RMSE	0.9699	0.9305	0.9247	0.9384	0.9837	1.2617

Table 3.2: Prediction performance of each time step before the fine-tuning

From these tables, we can clearly tell that the predictions of both models decrease

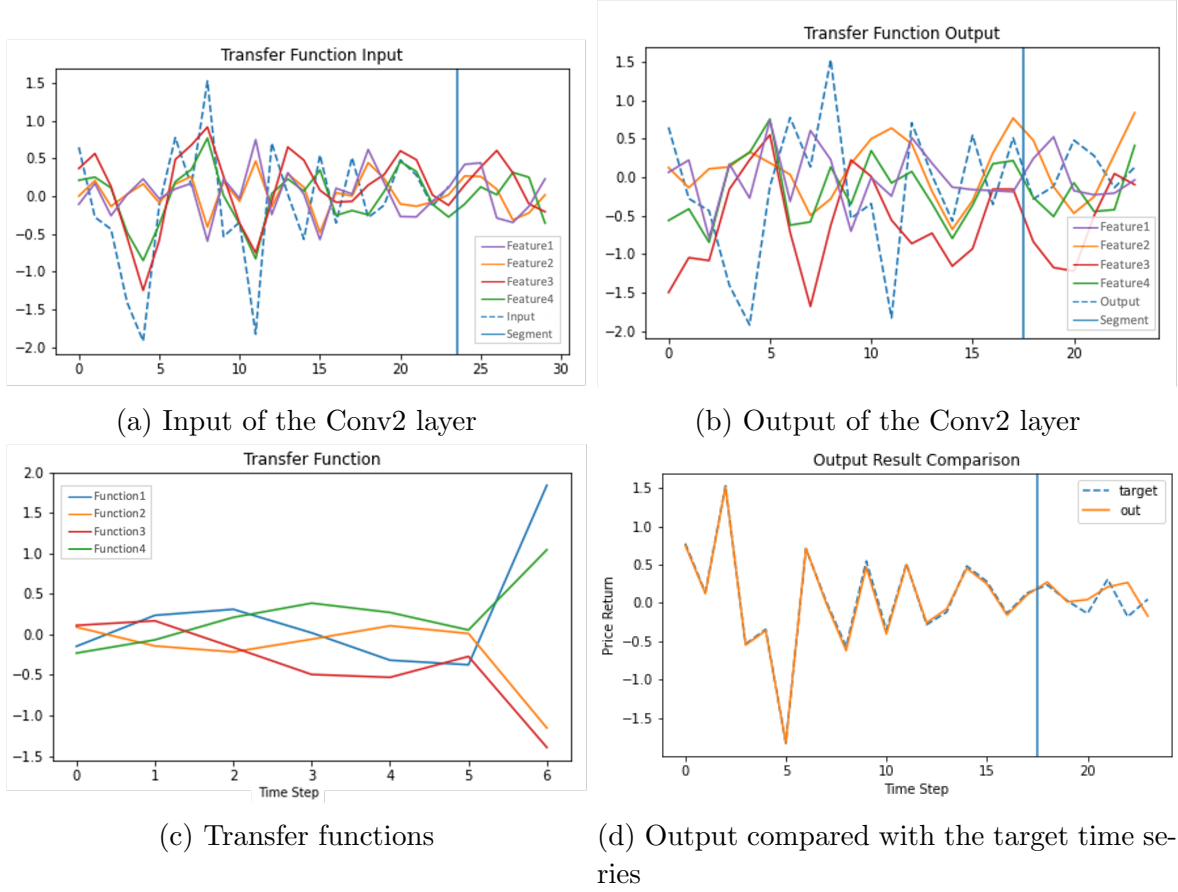


Figure 3.8: The behaviors of the model training in the second step

Time Step	1	2	3	4	5	6
RMSE	0.8815	0.8673	0.8972	0.9089	0.9720	1.2626

Table 3.3: Prediction performance of each time step after the fine-tuning

with the time step. The prediction is less accurate for the time points that are more away. This is reasonable because things are more unpredictable in the later future as we know less information about it, and it becomes more unpredictable.

Next, we compare our model with the model proposed by Chong et al. [16], which adopted the 3-layer multi-layer perceptions (MLP) model together with data pre-processing methods, such as principal component analysis (PCA), autoencoder (AE) and Restricted Boltzmann Model (RBM), to predict the price return of stocks from the South Korean stock market.

To align with the experiment setting of their model, we use the root mean-square error (RMSE) matrix to measure the performance. And select only the first element of our prediction for the calculation, as they only predict the value of the next time step.

The model before fine-tuning is also included for the comparison because it also has the predictive ability with the extended impact segment.

Model	Pre-fine-tuning	post-fine-tuning	Chong's
RMSE	0.9699	0.8815	0.9621

Table 3.4: Prediction performance comparison

Referring to Table. 3.4, we can see that our model already has considerable predictive capability before the prediction branch being trained. And the performance is already close to Chong's. After the fine-tuning of the prediction branch, the performance of the model is greatly improved and it surpasses Chong's model. The prediction error is 8.38% less. It proves the feasibility of the model structure, which has great prediction capability having an interpretable and reliable mechanism at the same time.

3.4.2 Performance of the Acceleration

The hardware is designed through high-level synthesis (HLS) and implemented on the Xilinx ZCU102 edge device powered by a Xilinx Zynq UltraScale+ MPSoC chip. The clock frequency is set to 200 MHz, while the dataset used to test the accelerator aligns with the one used for training.

Resources

The resource consumption of the accelerator is shown in Table. 3.5

Resources Usage	BRAM	DSP	FF	LUT
#	468	471	93581	100090

Table 3.5: Resource usage of the accelerator

Latency

Table. 3.6 shows the comparison of latency among different implementations. In order to quickly verify the computation of the model, I wrote another python model that mimics the computing flow of the Pytorch model. The latency of this model, named CPU python, is also tested. What's more, the CPU Pytorch and GPU Pytorch are Pytorch models running respectively on CPU and GPU. The speedup row indicates how much the FPGA accelerator improves the processing time.

Model	CPU python	CPU Pytorch	GPU Pytorch	FPGA
Latency (ms)	3.314	2.895	4.801	0.724
Speedup	4.577	3.999	6.631	-

Table 3.6: Latency comparison

From the table, we can see the FPGA latency is lower than all of the other implementations. FPGA implementation runs 3.999 times faster than the fastest CPU Pytorch, while 4.577 and 6.631 times faster than the CPU python model and the GPU Pytorch model. In the HFT competition, where every millisecond is critical, the improvement of the processing time enables the network model to be considered as a choice in the higher-frequency scenario.

Another remarkable point here is that the GPU model spends more time than other CPU models, which has conflict with our general understanding. It is because we are testing the latency by feeding the data one by one instead of in a large batch. The GPU experienced an overhead from the CUDA initialization operation, which happens every time at the beginning of the run. If the batch size is large, this overhead could be neglected. But we are feeding a single frame of data, so the overhead becomes apparent. However, frame-by-frame data feeding is common in the real application where market data are streaming continuously. It also reveals that though GPU has high throughput, it is not suitable for this application, since the input is fed one by one, and that the operation of a GPU requires the intervention of a CPU.

3.5 Conclusion

In this chapter, the predictive model together with its acceleration is proposed as part of the trading strategy module in the HFT trading system loop. The model consists of a transform-driven encoder part and a convolutional decoder part, which should be trained by two steps for first obtaining the market pattern and then predicting based on these patterns. And the behavior of the model is interpretable and the predicting accuracy is impressive. Then we deploy the model on the FPGA platform and apply some optimizing methods to speed up the computing. As a result, the latency is optimized and shorter than that on other generally used platforms. This work sheds light on the possibility of applying the neural network model in the HFT trading system.

CHAPTER 4

CONCLUSION AND EXPECTATION

4.1 Conclusion

The thesis elaborates on my two works as part of the HFT system. The memory-time balanced local order book reconstructing system and the accelerated interpretable predictive model are proposed.

The proposed local order book reconstructing system consisting of the fixed indexing space (FIS) and the dynamic management space (DMS) in the memory takes into account both the fast indexing searching and the memory reusing, which helps us to obtain the impressively low memory usage and competitive low latency. Besides, the four PS-PL interfaces are explored by implementing them as the order book memory access data path. And they show close performance in this application.

And in Chapter 3, we proposed a predictive model which is designed based on the interpretation of the market. The market signals are decomposed into the event signals and the event impact. The model automatically learns to extract the event signals and store different types of market impact series in the transfer function. After that, the model is trained by two steps: the first one is to recover the price signal in order to learn the ability to decompose the market signal and obtain the market impact pattern, while the second step shapes the prediction branch and enhances the predicting ability. The model shows the interpretability of its behaviors, while its accuracy is remarkable compared to others. In the end, the model is moved to the FPGA platform to speed up the execution, which enables the model to be applied in a high-frequency scenario.

These works require knowledge of reconfigurable system design, neural network design, data processing, and insight into the financial industry. Hoping in the future I can find more interesting topics regarding this area.

4.2 Future Work

In the first work, the proposed system is tested with the replayed historical data of the market. In the future, I hope to test the system by connecting it to the streaming API provided by the exchange. This is to verify the system under the real scenario and generate a stable system. And for the second work, the predictive model, I expect to train the model by using data from other exchanges and see if the pattern of the market still exists or how the pattern varies from market to market. And then use the model in a trading strategy to see how much profit the model can bring.

In addition to the works proposed here, I am also looking forward to working of other modules in the HFT trading system, such as the network module and the order generation module. And in the end make it a close loop system that can automatically receive market data, analyze the market and place orders from one end to another.

REFERENCES

- [1] Ruihong Huang and Tomas Polak. Lobster: Limit order book reconstruction system. *Available at SSRN 1977207*, 2011.
- [2] Conghui He, Haohuan Fu, Wayne Luk, Weijia Li, and Guangen Yang. Exploring the potential of reconfigurable platforms for order book update. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*, pages 1–8, September 2017. ISSN: 1946-1488.
- [3] Haohuan Fu, Conghui He, Wayne Luk, Weijia Li, and Guangwen Yang. A Nanosecond–Level Hybrid Table Design for Financial Market Data Generators. In *2017 IEEE 25th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 227–234, Napa, CA, USA, April 2017. IEEE.
- [4] Kenneth E Batchier. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*, pages 307–314, 1968.
- [5] Wei Huang, Yoshiteru Nakamori, and Shou-Yang Wang. Forecasting stock market movement direction with support vector machine. *Computers & operations research*, 32(10):2513–2522, 2005.
- [6] Muh-Cherng Wu, Sheng-Yu Lin, and Chia-Hsin Lin. An effective application of decision tree to stock trading. *Expert Systems with applications*, 31(2):270–274, 2006.
- [7] MV Subha and S Thirupparkadal Nambi. Classification of stock index movement using k-nearest neighbours (k-nn) algorithm. *WSEAS Transactions on Information Science & Applications*, 9(9):261–270, 2012.
- [8] Dalton Lunga and Tshilidzi Marwala. Online forecasting of stock market movement direction using the improved incremental algorithm. In *International Conference on Neural Information Processing*, pages 440–449. Springer, 2006.
- [9] Alex Sherstinsky. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica D: Nonlinear Phenomena*, 404:132306, 2020.

- [10] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [13] Hyejung Chung and Kyung-shik Shin. Genetic algorithm-optimized long short-term memory network for stock market prediction. *Sustainability*, 10(10):3765, 2018.
- [14] Fei Qian and Xianfu Chen. Stock prediction based on lstm under different stability. In *2019 IEEE 4th International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*, pages 483–486. IEEE, 2019.
- [15] Masaya Abe and Hideki Nakayama. Deep learning for forecasting stock returns in the cross-section. In *Pacific-Asia conference on knowledge discovery and data mining*, pages 273–284. Springer, 2018.
- [16] Eunsuk Chong, Chulwoo Han, and Frank C Park. Deep learning networks for stock market analysis and prediction: Methodology, data representations, and case studies. *Expert Systems with Applications*, 83:187–205, 2017.
- [17] Yuming Li, Pin Ni, and Victor Chang. An empirical research on the investment strategy of stock market based on deep reinforcement learning model. In *COMPLEXIS*, pages 52–58, 2019.
- [18] Alan V Oppenheim, Alan S Willsky, Syed Hamid Nawab, Gloria Mata Hernández, et al. *Signals & systems*. Pearson Educación, 1997.
- [19] Ju Rang Yan. Oscillation theorems for second order linear differential equations with damping. *Proceedings of the American Mathematical Society*, 98(2):276–282, 1986.
- [20] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of the thirteenth international conference on*

- artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.
- [21] Yong Dou, Stamatis Vassiliadis, Georgi Krasimirov Kuzmanov, and Georgi Nedeltchev Gaydadjiev. 64-bit floating-point fpga matrix multiplication. In *Proceedings of the 2005 ACM/SIGDA 13th international symposium on Field-programmable gate arrays*, pages 86–95, 2005.
- [22] Yufei Ma, Yu Cao, Sarma Vrudhula, and Jae-sun Seo. Optimizing the convolution operation to accelerate deep neural networks on fpga. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 26(7):1354–1367, 2018.

APPENDIX A

ORDER BOOK

Block Design diagram of Adopting Different Interfaces

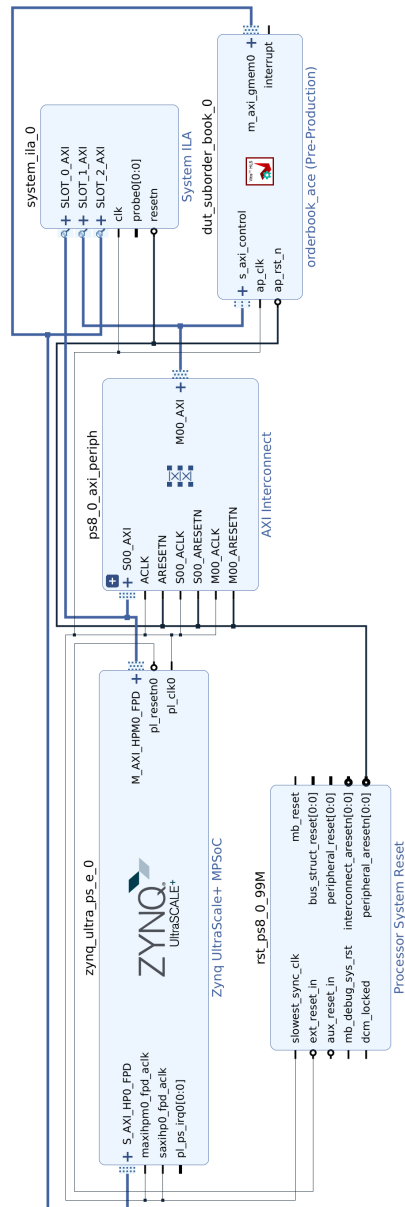


Figure A.1: Architecture adopting HP interface

